



Arquitectura Objetivo (TO-BE), Blueprint de Plataforma y Estrategia de Evolución

Modernización y Contenerización POS

MARZO 2026

1. Propósito de la Arquitectura Objetivo

El propósito de la Arquitectura Objetivo (TO-BE) es transformar el ecosistema distribuido, fuertemente acoplado y vulnerable de EDIMCA hacia una plataforma Cloud-Native, segura, observable y completamente desacoplada.

El objetivo no es realizar un simple *Lift & Shift* (mover las aplicaciones tal como están), sino rediseñar la solución para:

- Reducir el acoplamiento técnico y funcional (eliminando librerías monolíticas transversales como core-dto).
- Eliminar la obsolescencia crítica migrando a un stack moderno y ultra-eficiente, reemplazando Spring Boot / Java EE legacy por Spring Boot (optimizado para contenedores).
- Encapsular el ERP JD Edwards detrás de una capa de anticorrupción (ACL) para proteger los nuevos dominios.
- Habilitar el despliegue continuo y la escalabilidad elástica mediante orquestación de contenedores (Kubernetes/OpenShift o Amazon EKS Serverless).

2. Drivers de Negocio y Técnicos

Drivers de Negocio:

- Continuidad Operativa: Aislar dominios para que fallos locales no generen caídas globales (ej. un fallo en producción no detiene las ventas en POS).
- Escalabilidad y Flexibilidad: Soportar el crecimiento transaccional y un *Time-to-Market* acelerado para nuevas funcionalidades.
- Reducción de Riesgo de Cumplimiento: Mitigar brechas de seguridad críticas operando bajo estándares corporativos y auditables.

Drivers Técnicos:

- Adopción Serverless Kubernetes: Uso de orquestación avanzada (ej. AWS Fargate sobre Amazon EKS) para escalar recursos dinámicamente bajo demanda.
- Modernización del Stack Java: Transición hacia Spring Boot, un framework *Kubernetes-Native* de arranque ultrarrápido y bajo consumo de memoria.
- Identidad y Mensajería Moderna: Centralización con Keycloak y adopción de streaming de eventos (Kafka/MSK).

3. Problemas que el AS-IS debe resolver

La arquitectura TO-BE está diseñada para resolver de manera explícita y definitiva los siguientes problemas críticos identificados en el estado actual:

3.1. Problemas estructurales

- Monolitos distribuidos: El backend core POS concentra demasiado comportamiento, lógica de negocio y responsabilidades de distintos dominios.

- **Shared-Kernel rígido:** El módulo core-dto funciona como un kernel compartido rígido que crea un acoplamiento transversal severo entre casi todos los servicios, obligando a despliegues en cascada y bloqueando el versionado independiente.
- **Seguridad perimetral deficiente:** El Gateway y el componente Auth presentan controles de seguridad débiles y una implementación de autenticación *custom* no apta para un ecosistema empresarial moderno.
- **Duplicidad funcional:** Existen dos caminos de OCR (nuevo y legacy), lo que duplica la responsabilidad, el esfuerzo de mantenimiento y la complejidad operativa.

3.2. Problemas de seguridad

- **Gestión de Identidades Vulnerable:** Secreto JWT hardcodeado y expiración de tokens JWT no efectivos, lo que abre una ventana crítica para el secuestro de sesiones.
- **Fuga de Secretos:** Credenciales de bases de datos, claves de APIs y accesos de mensajería expuestos en archivos .properties, enums y utilitarios compartidos en los repositorios.
- **Superficie de ataque amplia:** Políticas de CORS abiertas (*) sin restricción de origen y endpoints administrativos (Spring Boot Actuator) expuestos públicamente en varios componentes.

3.3. Problemas de portabilidad y Cloud Readiness

- **Dependencia de infraestructura física:** Uso intensivo de rutas absolutas de disco, interfaces de red locales y dependencia estricta del *filesystem* del servidor host.
- **Red rígida:** Dependencia de servicios apuntando a localhost y de hosts/IPs embebidos rígidamente en el código fuente, lo que impide el balanceo de carga moderno.
- **Utilitarios no portables:** Uso de utilitarios (ej. generación PDF/OCR) que asumen la existencia de un entorno de servidor tradicional y la ejecución local de binarios instalados en el SO.

3.4. Problemas de observabilidad y operación

- **Calidad de software incierta:** Baja cobertura de pruebas automatizadas o pruebas prácticamente ausentes en los repositorios más críticos, elevando el riesgo de regresión en cada paso a producción.
- **Cajas negras operativas:** Observabilidad no estandarizada; en varios componentes no se evidencian métricas estructuradas, trazas distribuidas ni un modelo de salud (*health model*) apto para la auto-recuperación en orquestadores.

4. Principios y Estándares de Arquitectura Objetivo

La nueva arquitectura establece una definición formal multinivel. Para evitar ambigüedades y regir el futuro desarrollo del ecosistema, se establecen como base los siguientes pilares de diseño:

Estilo arquitectónico definido:

- **Arquitectura de microservicios basada en Domain-Driven Design (DDD):** Separación estricta de las responsabilidades funcionales.

- **Arquitectura hexagonal (Ports & Adapters) como patrón interno estándar:** Aislamiento del dominio central frente a bases de datos, APIs o colas de mensajería.
- **Arquitectura orientada a eventos (Event-Driven Architecture):** Sustitución del acoplamiento síncrono por la publicación y consumo de eventos inmutables.

4.1. Principios de diseño

- **Domain-first:** La separación principal de los microservicios se hará por dominios de negocio reales (Ventas, Producción, Pricing, Documentos), no por tecnologías o silos de equipos.
- **Micro-frontends (Desacoplamiento End-to-End):** La separación por dominios no aplica solo al backend. Las interfaces de usuario web monolíticas (SPA) se modularizarán utilizando una arquitectura de **Micro-frontends**, permitiendo despliegues independientes por cada dominio (ej. un micro-frontend autónomo para POS, otro para Producción) integrados de forma transparente para el usuario final en un contenedor principal o *App Shell*.
- **API-first:** Los contratos (REST/OpenAPI) y los esquemas de eventos (AsyncAPI) deben definirse, documentarse y acordarse antes de iniciar cualquier implementación de código.
- **Loose coupling (Bajo acoplamiento):** Ningún componente de negocio debe depender de un *shared-kernel* rígido (como *core-dto*) para evolucionar.
- **Stateless by default:** Los microservicios serán inherentemente *stateless*. Todo estado persistente quedará almacenado en *stores* desacoplados.
- **Evolución Incremental (Strangler Pattern):** Se descarta una reescritura *Big Bang*. La evolución será progresiva; la migración se realizará **estrictamente por Dominio de Negocio (DDD)**. A medida que un nuevo dominio se refactoriza, el API Gateway estrangulará el tráfico del monolito AS-IS hacia los nuevos microservicios TO-BE.
- **Security by design:** La identidad, autorización, gestión de secretos y el cifrado en tránsito (mTLS) se diseñan como capacidades nativas.
- **Observability-first:** Logs, métricas, *health checks*, trazas y auditoría se instrumentan desde el primer día.

4.2. Principios de integración y Rol de JD Edwards

- **JDE como Legacy System of Record:** El ERP JD Edwards se mantendrá como sistema *legacy* central y **no sufrirá modificaciones ni modernizaciones** en su código fuente actual.
- **Aislamiento Total mediante ACL:** La Capa de Anticorrupción (Anti-Corruption Layer) será el **único y exclusivo** punto de integración. Encapsulará a JD Edwards para proteger a los nuevos microservicios (y sus modelos DDD) de contaminarse con los contratos SOAP anticuados y la deuda técnica del ERP.
- **Separar sincronicidad:** Separar estrictamente las integraciones síncronas (consultas UI) de las asíncronas (comandos de estado, sincronización de maestros).
- **Punto de acceso unificado:** Evitar el acceso directo a bases de datos corporativas desde múltiples microservicios.

4.3. Principios de datos

- Evitar DTO shared global: Prohibición del uso de librerías DTO compartidas masivamente.
- Contratos versionados: Usar contratos versionados por dominio (ej. v1, v2).
- Database-per-Service: Implementar una base de datos exclusiva por servicio.
- Publicación de Eventos: Publicar eventos de negocio con significado funcional, no objetos crudos internos.

5. Dominios Funcionales y Bounded Contexts

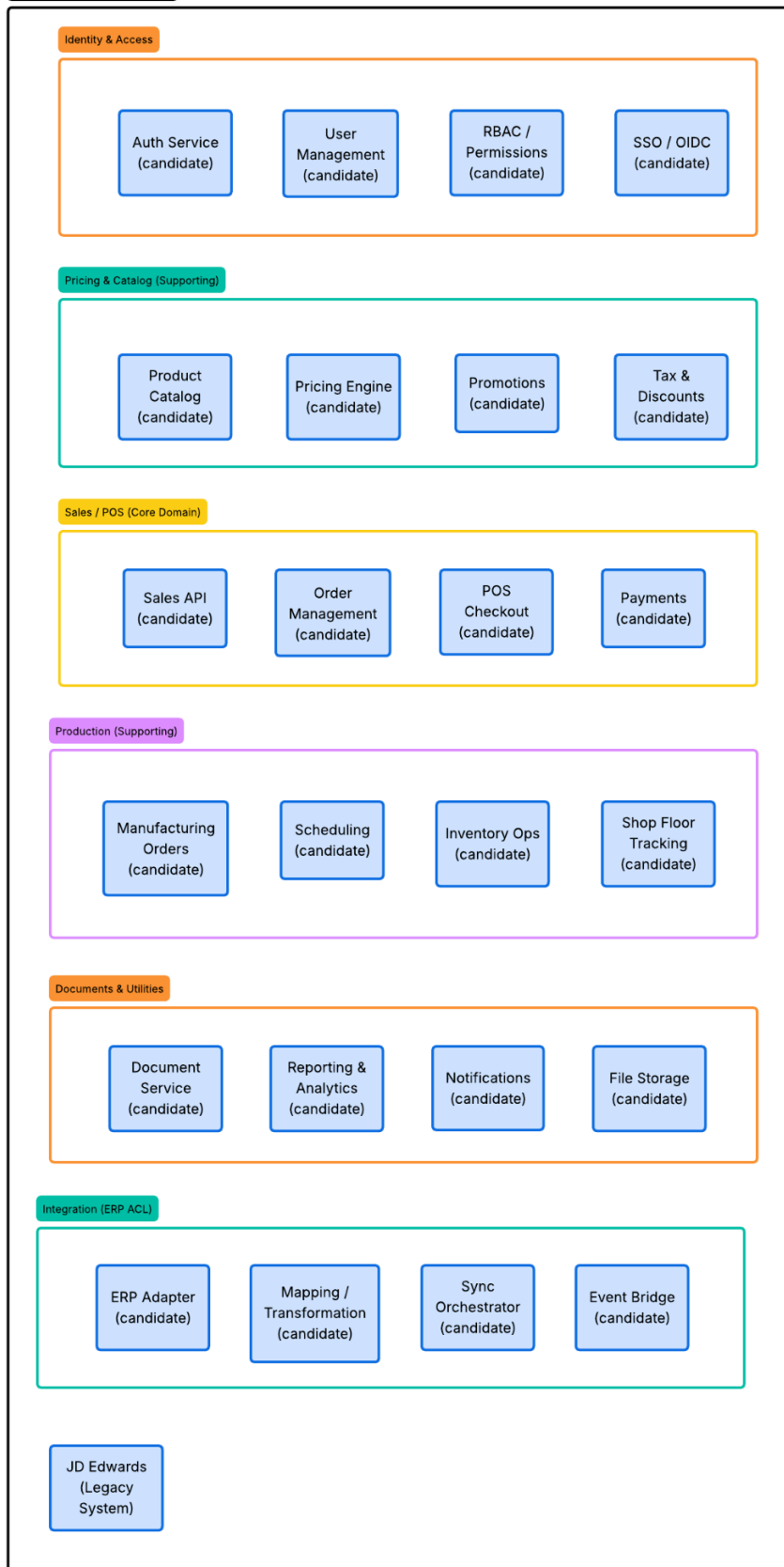
Los dominios funcionales se han identificado y definido a partir del análisis profundo de los **procesos de negocio reales y la delimitación de sus *bounded contexts* correspondientes**. Esta visión permite aislar correctamente las responsabilidades sin arrastrar la deuda técnica actual.

Dentro de este modelado, el sistema **POS no es un subdominio**, sino que se erige explícitamente como el **Dominio Core (Core Domain)** del ecosistema, al ser el pilar fundamental y diferenciador para la facturación y la operatividad comercial.

Se reestructura el negocio eliminando los monolitos y organizando las capacidades en dominios cohesivos:

1. **Sales & POS (Dominio Core):** Órdenes, facturación y recaudos de clientes en tienda física.
2. **Production (Dominio de Soporte):** Control de producción, transformaciones en taller, máquinas y operadores.
3. **Pricing & Catalog (Dominio de Soporte):** Listas de precios corporativas, stock y maestros de productos.
4. **Documents & Utilities (Dominio Genérico):** Extracción OCR de montos, generación PDF y notificaciones.
5. **Integration ERP (Dominio de Infraestructura):** Capa ACL dedicada exclusivamente a conversar con JD Edwards.
6. **Identity & Access (Dominio de Infraestructura):** Autenticación centralizada y perfiles.

POS-Centric DDD Context Map



6. Microservicios Candidatos (Propuesta de División y Refactor)

La evolución de los componentes AS-IS hacia microservicios se realizará de manera incremental, priorizando dominios de negocio y utilizando el patrón Strangler, evitando en todo momento reescrituras masivas que arriesguen la operación.

Componente AS-IS	Artefactos Objetivo (TO-BE divididos)	Tecnología Target Requerida
edimca-production-core	prod-order-svc, prod-operator-svc, prod-machine-svc	Spring Boot (Java 17+), PostgreSQL
edimca-pos-core	pos-order-svc, pos-payment-svc, pos-customer-svc	Spring Boot (Java 17+), PostgreSQL
edimca-process-batch	Tareas nativas CronJob de Kubernetes	Spring Boot (Modo CLI/Batch)
edimca-pos-sincro	jde-acl-adapter (Capa Anticorrupción)	Spring Boot , Consumidor Kafka
edimca-core-dto	Eliminación Total.	OpenAPI y modelos locales
auth-project-pd149	Identity Provider Server	Keycloak (Imagen Oficial)
pdf-project / OCR	pdf-generator-svc, ocr-processor-svc	Node.js 20 LTS / Python 3.11
edimca-pos-frontr / production-front	pos-mfe, production-mfe, app-shell	React Moderno / Vite / Module Federation

7. Alternativas de Arquitectura de Solución y Plataformas

Se proponen vías de evolución arquitectónica (AWS, On-Premise e Híbrido). De estas opciones, **AWS es la alternativa recomendada** al ser una arquitectura Cloud-Native completamente gestionada. La opción **On-Premise es el equivalente funcional**, pero operado íntegramente por el cliente.

Cabe recalcar enfáticamente que **ambas arquitecturas mantienen equivalencia funcional en seguridad, observabilidad, CI/CD y resiliencia**.

7.1. Alternativa 1: Arquitectura Cloud-Native en AWS (Recomendada)

Delega el mantenimiento de la infraestructura a servicios gestionados por AWS, minimizando la carga operativa y maximizando la escalabilidad.

Ventajas de la Nube (AWS) para EDIMCA:

La adopción de una arquitectura Cloud-Native en AWS provee beneficios tangibles que justifican la inversión tecnológica:

1. Reducción de la Carga Operativa (*Undifferentiated Heavy Lifting*): Al utilizar Amazon EKS con perfiles Fargate (Serverless), el equipo de infraestructura de EDIMCA ya no debe provisionar, parchear ni mantener sistemas operativos o máquinas virtuales subyacentes.
2. Elasticidad Real y Pago por Uso: Los componentes *stateless* escalan horizontalmente de forma automática según el tráfico del POS y se contraen en horas no laborables, optimizando costos.
3. Alta Disponibilidad Nativa: La arquitectura se despliega a través de múltiples Zonas de Disponibilidad (Multi-AZ) dentro de una región de AWS, garantizando resiliencia ante caídas de centros de datos completos sin esfuerzo de configuración manual.
4. Seguridad Intrínseca e Integrada: Cifrado en tránsito y en reposo gestionado nativamente (AWS KMS), y protección perimetral de grado militar mediante AWS WAF y Shield contra ataques DDoS y vulnerabilidades web.

Capacidad Requerida	Servicio Recomendado AWS	Descripción del Servicio
Orquestación	Amazon EKS con AWS Fargate	K8s gestionado. Fargate permite correr contenedores <i>Serverless</i> .
Mensajería	Amazon MSK / Amazon SQS	Kafka Gestionado para eventos. SQS/SNS para colas asíncronas.
Bases de Datos	Amazon RDS / Aurora PostgreSQL	BD relacionales gestionadas (Multi-AZ y parches delegados).
Almacenamiento	Amazon S3	Object Storage para PDFs, comprobantes OCR e imágenes.
Seguridad / IAM	Amazon EKS + Keycloak	Keycloak en EKS para OIDC. <i>AWS Secrets Manager</i> para secretos.
API Gateway & WAF	AWS ALB + AWS WAF	Balanceador L7 protegido por el Firewall Web de AWS.

DevOps y CI/CD	AWS CodePipeline / ECR	Registro de contenedores con escaneo de CVEs y pipelines CI/CD.
----------------	------------------------	---

7.2. Alternativa 2: Arquitectura On-Premise Modernizada (OpenShift / Kubernetes)

Mantiene el control físico del hardware dentro de los centros de datos de EDIMCA, modernizando la capa de virtualización hacia orquestación de contenedores.

Capacidad Requerida	Plataforma Recomendada	Costo / Modalidad
Orquestación K8s	Red Hat OpenShift	Costo (Suscripción Enterprise)
Orquestación K8s (Alt)	RKE2 / Vanilla K8s	Free (Open Source)
Mensajería (Kafka)	Strimzi (Apache Kafka)	Free (Open Source)
Bases de Datos	PostgreSQL (CrunchyData / Patroni)	Free / Soporte de Pago
Almacenamiento Objeto	MinIO	Free (AGPL) / Enterprise
Seguridad / IAM	Keycloak / HashiCorp Vault	Free (Open Source / CE)
DevOps (GitOps)	GitLab + ArgoCD	Free (CE) / Enterprise

7.3. Alternativa 3: Enfoque Híbrido (Arquitectura de Transición)

Es totalmente viable adoptar un enfoque híbrido, especialmente útil y estratégico durante la fase de transición de la arquitectura:

- **Ejecución Distribuida:** Los microservicios *stateless* (Spring Boot, Node, Frontend) y el Gateway de seguridad se despliegan en la nube (AWS EKS), aprovechando la elasticidad y el WAF.
- **Datos Resguardados:** El ERP JD Edwards y las bases de datos relacionales heredadas permanecen On-Premise.
- **Conectividad Segura:** La comunicación se orquesta mediante un túnel seguro (AWS Site-to-Site VPN o AWS Direct Connect), garantizando que el *Anti-Corruption Layer (ACL)* en AWS interactúe con el JDE local sin exponer el ERP a internet.

7.4. Madurez y Evaluación de Alternativas

- Ambas alternativas (AWS y On-Premise) proponen una arquitectura final con **capacidades y madurez equivalentes**. Las tecnologías listadas en AWS son referenciales y recomendadas, pero no obligatorias si se opta por el stack On-Premise.
- Sin embargo, **AWS tiene prioridad estratégica**. El enfoque On-Premise demandará un alto grado de madurez operativa (SRE/DevOps avanzado) y un esfuerzo considerable por parte del equipo interno de EDIMCA para operar, parchear y escalar clústeres de Kafka (Strimzi),

HashiCorp Vault, bases de datos en HA (Patroni) y el propio orquestador Kubernetes sin el soporte nativo de servicios administrados. AWS delega esta carga operativa (IaaS/PaaS) permitiendo a EDIMCA enfocarse exclusivamente en el desarrollo de valor.

8. Seguridad Objetivo y Gestión de Accesos (Zero Trust)

La arquitectura de seguridad adopta un modelo de "Confianza Cero" mediante la autenticación obligatoria en cada *request*.

8.1. Identidad y Acceso (IAM Central)

Se propone Keycloak como plataforma de identidad central, proveyendo:

- Estándares OIDC / OAuth2 y Single Sign-On (SSO).
- Gestión centralizada de roles, grupos y *Client Credentials* para flujos M2M.
- *Service accounts* y federación futura.

8.2. Despliegue de Keycloak

Keycloak se desplegará en EKS/K8s bajo estrictos lineamientos:

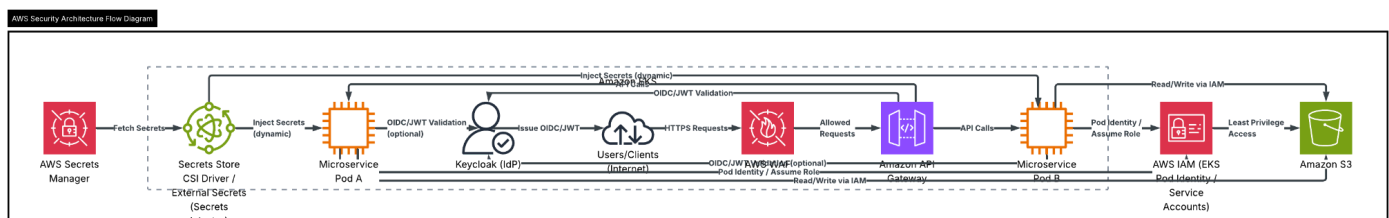
- En un namespace dedicado, con Alta Disponibilidad (HA).
- Usando PostgreSQL administrado (Amazon RDS/Aurora).
- Expuesto mediante un Ingress seguro y respaldado por secretos administrados.

8.3. Seguridad de Workloads

- Tokens JWT: Firmados y con expiración estricta (TTL).
- Validación: Efectuada en el Gateway y/o mediante políticas Sidecar.
- AWS WAF: Protección en el borde contra vulnerabilidades web.
- IAM (EKS Pod Identity): Asignación de permisos AWS vía *Service Accounts* (Least Privilege). En EKS Auto Mode no requiere instalar agentes manualmente.

8.4. Gestión de Secretos

- AWS Secrets Manager o HashiCorp Vault. Eliminación total de secretos embebidos en repositorios, imágenes o archivos *.properties*.

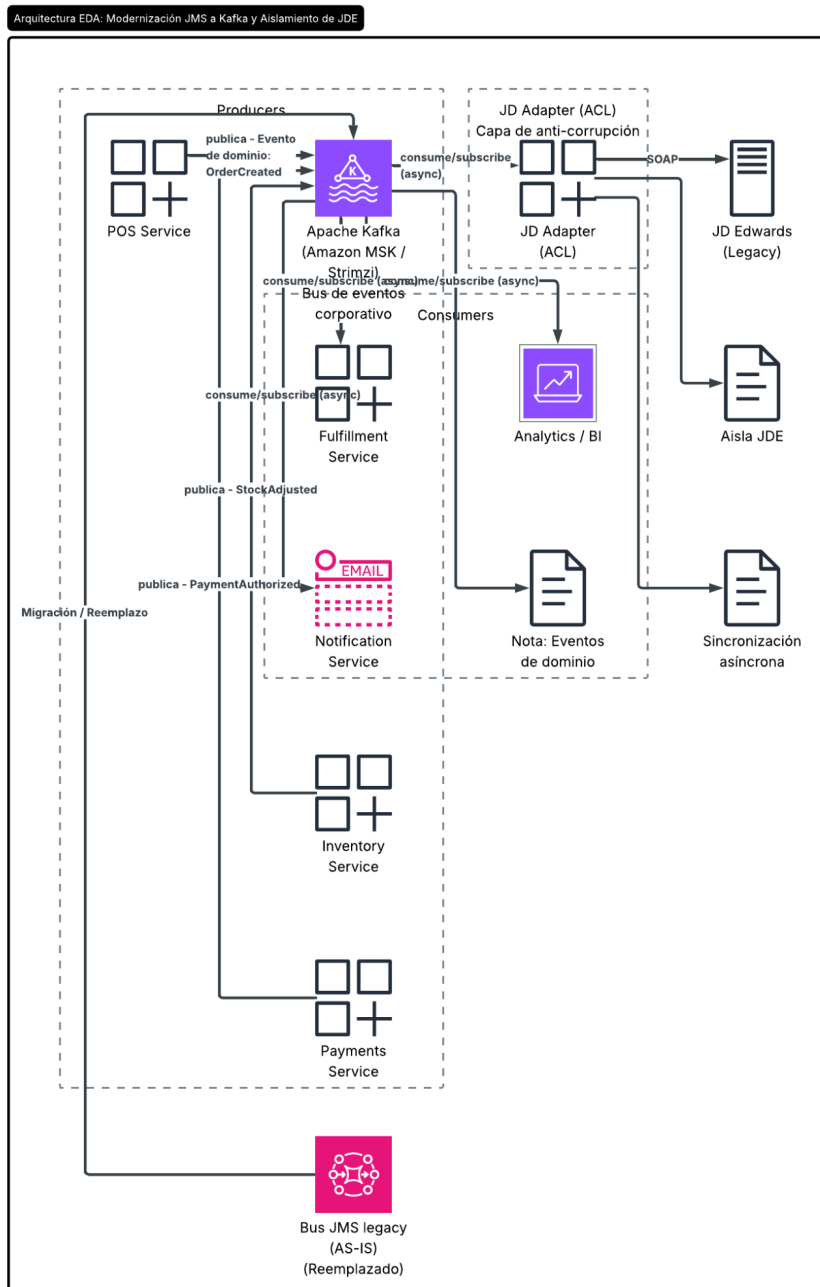


9. Lineamientos de Integración y Mensajería

Se abandona el modelo de integración directa a favor de una **Arquitectura Orientada a Eventos**

(EDA).

- **Eventos de Negocio:** Publicación de hechos inmutables (ej. InvoiceGenerated, OrderApproved).
- **Sustitución de JMS Legacy:** Migración inmediata del broker ActiveMQ altamente vulnerable hacia **Apache Kafka** (Amazon MSK en la nube o Strimzi en On-Premise).
- **Aislamiento del ERP: JD Edwards no será modificado y será encapsulado mediante una Anti-Corruption Layer (ACL).** El ERP operará detrás de un muro virtual. El microservicio jde-acl-adapter consumirá eventos desde Kafka, los transformará al esquema del ERP y ejecutará las llamadas SOAP necesarias. JDE nunca interactuará directamente con los nuevos microservicios.



10. Observabilidad Objetivo (Observability by Design)

Para reducir el MTTR y eliminar las cajas negras, se adopta un stack de telemetría moderno:

10.1. Componentes del Stack

- Métricas: Amazon Managed Service for Prometheus (AMP), compatible con PromQL para entornos de contenedores.

- Dashboards: Amazon Managed Grafana, para visualización unificada.
- Logs: OpenTelemetry (OTel) Collector transmitiendo hacia Amazon CloudWatch Logs y/o Amazon OpenSearch Service.
- Trazas: Instrumentación OpenTelemetry en Spring Boot, enviando trazas a AWS X-Ray o Grafana Tempo.
- Alertas: AMP + Grafana disparando eventos hacia Amazon SNS.

10.2. Recomendación de Logs para EDIMCA

- Todo microservicio emitirá logs estructurados en JSON.
- Almacenamiento primario en CloudWatch Logs. Analítica avanzada opcional con OpenSearch Service (aprovechando OpenSearch Ingestion para normalizar logs).

II. DevOps y Cadena de Despliegue (DevSecOps)

11.1. Propuesta DevOps y Herramientas

- Repositorios Git: Código versionado y limpio.
- Pipeline CI: Pruebas, *lint*, escaneo de vulnerabilidades (SAST/SCA) y *build* de imagen Docker.
- Amazon ECR: Almacenamiento seguro de imágenes.
- AWS CodeBuild & CodePipeline: Orquestación de despliegue hacia EKS (soportando clústeres privados).
- Despliegue GitOps: Uso de Helm/Kustomize aplicados vía ArgoCD.

11.2. Estrategia Recomendada

- Branching Model: *Trunk-based development* o GitFlow simplificado.
- Promociones: DEV → QA → PROD (Imágenes inmutables).
- Aprobaciones: Manuales para producción; despliegues *Canary* o *Blue/Green*.
- Seguridad Shift-Left: Escaneo obligatorio de secretos (Policy-as-Code).

12. Roles y Matriz RACI del Proyecto de Modernización

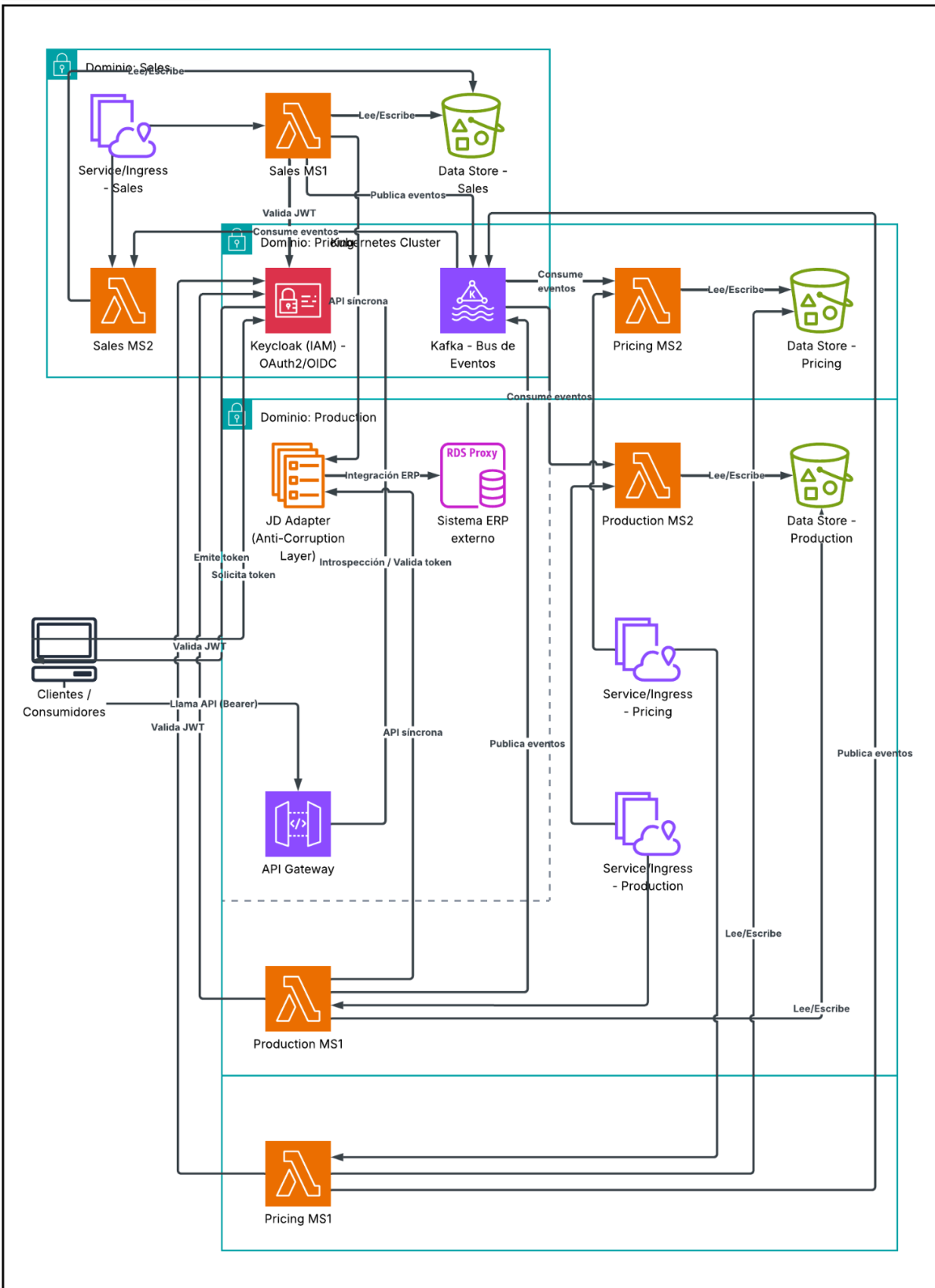
Fase / Actividad de Modernización	Arquitecto (ARQ)	Tech Lead (TL)	Developer (DEV)	DevSecOps (DSO)	Product Owner (PO)
Definición de Bounded Contexts y Contratos OpenAPI	A	R	R	C	C
Setup de Infraestructura Base (EKS, MSK, RDS)	A	C	I	R	I
Refactor de Monolito a Microservicios Spring Boot	C	A	R	C	I
Implementación de Seguridad (Keycloak, IAM, Secrets)	A	C	I	R	I
Creación de Pipelines CI/CD y GitOps	C	C	I	R/A	I
Pruebas de Integración e Ingeniería de Caos	C	A	R	R	I
Despliegue a Producción (Canary/Strangler)	C	R	I	R/A	C
(R: Responsible, A: Accountable, C: Consulted, I: Informed)					

13. Arquitectura Lógica Objetivo

La arquitectura lógica propuesta se estructura en capas concéntricas y dominios aislados, promoviendo el desacoplamiento, la alta cohesión y la independencia de despliegue:

- **Capa de Canales (Micro-frontends):** Transición de las interfaces web monolíticas hacia una **Arquitectura de Micro-frontends (MFE)**. Se desplegará un contenedor principal (*App Shell*) que orquesta y ensambla de forma transparente para el usuario final los distintos frontends de dominio (POS MFE, Producción MFE), hospedados de forma estática y desacoplada (ej. S3 + CloudFront).
- **Capa Perimetral (Ingress & Gateway):** Punto único de entrada de tráfico protegido por reglas de firewall (WAF) y balanceo de carga. Enruta las peticiones hacia los microservicios correctos o hacia el Identity Provider.
- **Capa Transversal de Seguridad (IAM):** **Keycloak** actúa como el guardián transversal de la identidad, emitiendo tokens OIDC/JWT para asegurar que toda petición hacia la plataforma esté debidamente autenticada y autorizada.
- **Capa de Microservicios por Dominio:** Servicios altamente granulares desarrollados en **Spring Boot** (ej. pos-order-svc, prod-operator-svc). Diseñados bajo el principio de *Database-per-Service*, garantizando la independencia total de los datos.
- **Capa de Eventos (Event Bus):** Apache Kafka (Amazon MSK o Strimzi) actúa como la columna vertebral asíncrona de la arquitectura. Permite que los dominios reaccionen a eventos de negocio (ej. InvoiceGenerated) sin acoplamiento temporal ni dependencias REST bloqueantes.
- **Capa Anticorrupción (ACL):** El *jde-acl-adapter* aísla al ecosistema moderno del ERP legacy (JD Edwards). Traduce los modelos de dominio locales a las estructuras SOAP/JMS que el ERP requiere, protegiendo a los microservicios de la deuda técnica externa.
- **Capa de Datos Independiente:** Uso de bases de datos PostgreSQL dedicadas por dominio, eliminando el problema de esquemas compartidos y bloqueos de tablas transversales.

Arquitectura de Microservicios en Kubernetes

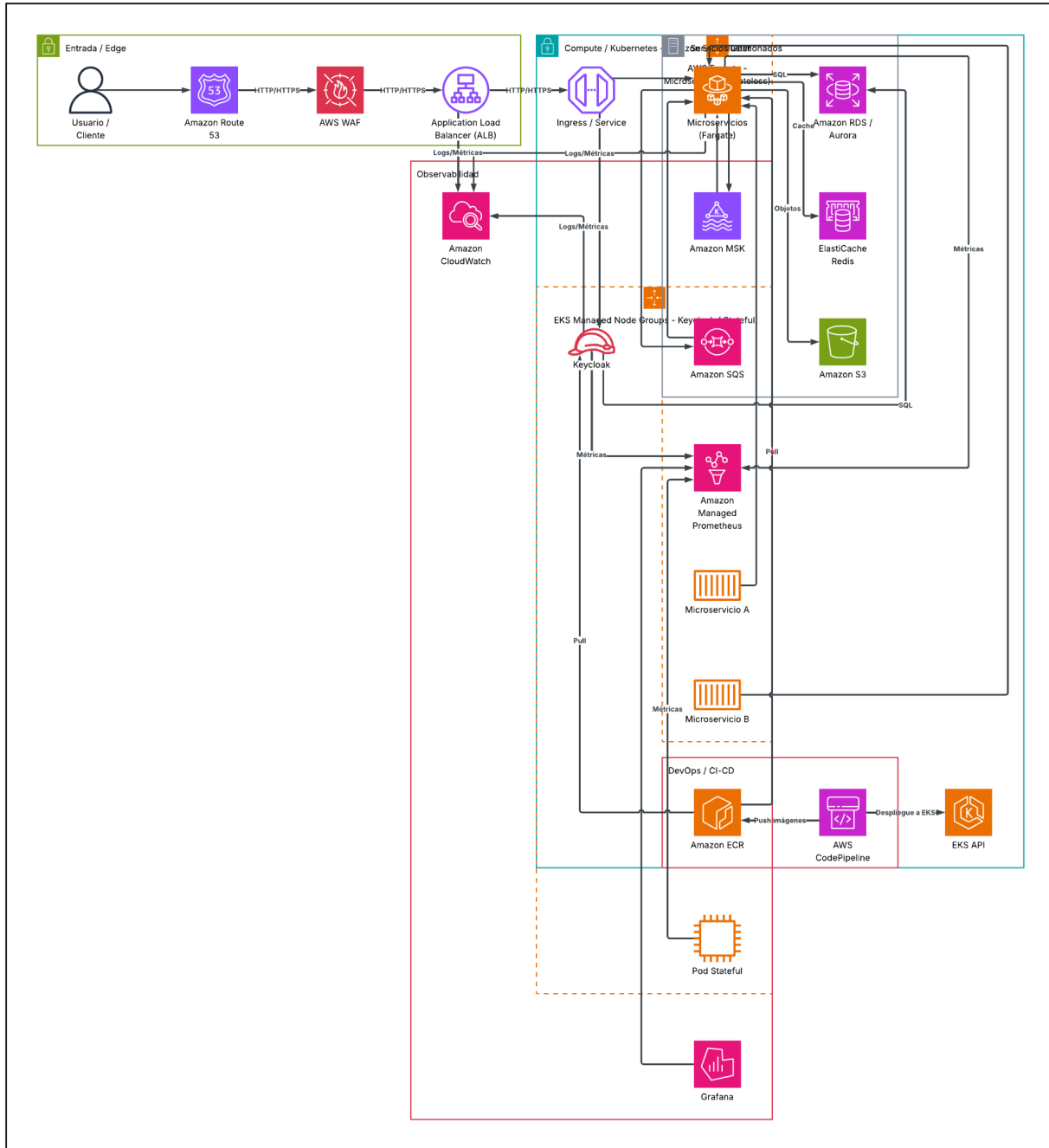


14. Arquitectura de Despliegue Objetivo en AWS

El despliegue físico-virtual en AWS se fundamenta en el uso de servicios administrados para minimizar drásticamente la carga operativa, delegar el mantenimiento de la infraestructura base y garantizar Alta Disponibilidad (HA) de forma nativa:

- **Red y Perímetro (VPC Multi-AZ):** La infraestructura reside en una Virtual Private Cloud (VPC) distribuida en al menos dos Zonas de Disponibilidad (AZs). El tráfico de los usuarios ingresa a través de Amazon Route 53 (DNS), atraviesa el AWS WAF (filtrado de amenazas y protección DDoS), y llega al Application Load Balancer (ALB) ubicado en las subredes públicas.
- **Cómputo y Orquestación (Amazon EKS):** El corazón de la ejecución es el clúster de Kubernetes gestionado por AWS, desplegado en las subredes privadas.
 - **AWS Fargate (Serverless):** Aloja los microservicios de negocio desarrollados en Spring Boot que son estrictamente *stateless*. Fargate escala los *pods* bajo demanda sin necesidad de que EDIMCA administre servidores o instancias EC2 subyacentes.
 - **Managed Node Groups (o EKS Auto Mode):** Alojan cargas de trabajo *stateful* o herramientas de plataforma pesadas (como Keycloak, Ingress Controllers o agentes de recolección de logs) que requieren control sobre el host o almacenamiento persistente atado (EBS).
- **Servicios Administrados de Datos y Mensajería:**
 - **Amazon RDS / Aurora PostgreSQL:** Proveen los motores de bases de datos relacionales con replicación nativa Multi-AZ, respaldos automáticos (snapshots) y mantenimiento delegado.
 - **Amazon MSK:** Cluster de Kafka totalmente gestionado que actúa como el bus de eventos corporativo, eliminando el mantenimiento crítico asociado a brokers de mensajería (mitigando los CVEs actuales de ActiveMQ).
 - **Amazon ElastiCache (Redis):** Manejo de cachés distribuidas de alta velocidad para reducir la carga sobre las bases de datos.
 - **Amazon S3:** Almacenamiento de objetos duradero (99.999999999%) para los documentos, reemplazando la frágil dependencia actual hacia el *filesystem* local para guardar PDFs e imágenes OCR.
- **DevOps y Observabilidad (Herramientas Core):** Integración nativa con la suite de AWS para el ciclo de vida. Amazon ECR actúa como el registro seguro de imágenes inmutables. AWS CodePipeline/CodeBuild facilitan el CI/CD, mientras que el stack de Amazon Managed Prometheus (AMP), Managed Grafana y CloudWatch Logs garantizan una telemetría y diagnóstico integral.

AWS Cloud-Native Deployment Diagram



ANEXOS ESTRATÉGICOS

ANEXO B1. Matriz Componente Actual (AS-IS) ↔ Componente Objetivo (TO-BE)

Para estructurar correctamente la estrategia de descomposición (DDD), los componentes se clasifican según su tipo de dominio:

Estrategia y Clasificación por Dominios (DDD)

Dominio	Estrategia DDD
POS & Ventas	Core Domain (El diferencial del negocio)
Production	Supporting Domain (Soporte a la operación)
Pricing & Catalog	Supporting Domain
Integration	ACL Domain (Anti-Corruption Layer)

Evolución Técnica por Componente:

Rol Funcional	Componente Actual (AS-IS)	Nueva Arquitectura (TO-BE)
Canales / Interfaz UI	SPAs Monolíticas (edimca-pos-frontr)	Micro-frontends (MFE) orquestados por un App Shell.
Seguridad / IAM	auth-project (Custom JWT vulnerable)	Keycloak (OIDC/OAuth2) en EKS/K8s.
Enrutamiento	edimca-gateway-service	AWS ALB + Ingress Controller.
Microservicios Core	Monolitos pos-core, production-core	Microservicios granulares en Spring Boot .
Contratos	Librería Monolítica edimca-core-dto	Retiro total . OpenAPI/Swagger por servicio.
Mensajería Asíncrona	Artemis / ActiveMQ (Con CVEs)	Amazon MSK (Kafka) o Strimzi On-Premise.
Procesos Batch	edimca-process-batch	Tareas efímeras vía CronJob de K8s.
Storage Documental	Filesystem local (edimca-repository)	Amazon S3 o MinIO.

ANEXO B2. Glosario de Términos Arquitectónicos y Mejores Prácticas

Término	Descripción Técnica en el Contexto de EDIMCA
Cloud Native	Enfoque de construcción de aplicaciones que explotan la nube (escalabilidad, resiliencia). Incluye contenedores, K8s e infraestructura declarativa.
Arquitectura Hexagonal	Patrón que aísla el núcleo de la lógica de negocio de las tecnologías externas (Bases de datos, UI, Mensajería) mediante puertos y adaptadores.
Domain-Driven Design (DDD)	Diseño de software que modela los sistemas según el dominio de negocio, definiendo <i>Bounded Contexts</i> y separando responsabilidades estrictamente.
Micro-frontends	Extensión del patrón de microservicios al frontend. Permite dividir una aplicación web monolítica en múltiples aplicaciones más pequeñas y autónomas (MFE), desarrolladas y desplegadas independientemente por equipos distintos, que se ensamblan en tiempo de ejecución (ej. Module Federation).
Stateless (Sin Estado)	Principio donde una aplicación no guarda contexto en disco local. Permite destruir y levantar contenedores libremente sin pérdida de datos.
12-Factor App	Metodología cloud-ready. Exige externalizar configuración, separar <i>Build/Run</i> , declarar dependencias y mantener inmutabilidad de despliegues.
GitOps	Práctica de usar Git como única fuente de verdad. Herramientas como ArgoCD sincronizan el clúster (EKS) con el repositorio Git de forma declarativa.

Observability by Design	Incorporación de métricas, logs JSON y trazas OpenTelemetry desde el código para acelerar el <i>troubleshooting</i> (MTTR).
Zero Trust Security	Modelo que exige autenticación obligatoria y validación en cada request, sin confiar por defecto en la red interna.

ANEXO B4. Dimensionamiento e Infraestructura Propuesta vs Carga Actual

El siguiente anexo establece una comparativa del dimensionamiento (Sizing) del entorno actual (AS-IS) versus el modelo propuesto (TO-BE), evidenciando cómo la nueva arquitectura gestiona los recursos operativos tanto para Producción (PROD) como para Certificación (QA).

La transición desde un modelo de asignación estática de Máquinas Virtuales (VMs) hacia un modelo de Pods dinámicos permite eliminar la subutilización de hardware y la colisión de servicios (servicios críticos compartiendo el mismo nodo).

B4.1 Análisis Comparativo de Carga (PROD y QA)

Componente Lógico	Estado AS-IS (Carga Actual)	Propuesta TO-BE (AWS Cloud-Native)	Propuesta TO-BE (On-Premise K8s/OpenShift)
Frontends (React/Angular)	PROD: 2 Nodos VM (CIR-POS-FRONT1/2). QA: 1 Nodo VM (CIR-QA-FRONT1). <i>Problema:</i> Asignación fija, recursos ociosos.	PROD / QA: Alojamiento <i>Serverless</i> en Amazon S3 + CloudFront. Costo casi nulo en QA y escalabilidad global infinita en PROD sin administrar servidores.	PROD: Despliegue en 2+ Pods (Nginx) balanceados vía Ingress. QA: 1 Pod (Escalable a cero si no hay uso).
Backends Core & APIs	PROD: 3 Nodos VM compartidos para Core, Batch, Auth y Gateway (CIR-POS-BACKEND1/2, APP). QA: Nodos compartidos de menor capacidad. <i>Problema:</i> Colisión de recursos (Cascade Failure).	PROD / QA: Despliegue en AWS Fargate (EKS). PROD: Auto-escalado Horizontal de Pods (HPA) basado en métricas (CPU/Mem). QA: Los Pods se escalan a 0 fuera del horario laboral, reduciendo costos.	PROD / QA: Despliegue en <i>Worker Nodes</i>. Aislamiento estricto por <i>Namespaces</i> y límites de recursos (Quotas) por cada microservicio Spring Boot.
Mensajería (Broker)	PROD: 2 Nodos VM Artemis (CIR-Atermis1/2). QA: Broker compartido/limitado.	PROD: Clúster Amazon MSK (Multi-AZ Provisioned). QA: Clúster MSK <i>Serverless</i> o nodo único para desarrollo.	PROD: Operador Strimzi gestionando un clúster Kafka de 3 Pods. QA: Nodo único de Kafka administrado por Strimzi.

Bases de Datos	PROD: 3 Nodos VM (Master, Réplica, PGPool). QA: Instancia única PostgreSQL.	PROD: Amazon Aurora PostgreSQL (Serverless v2 o Provisionado Multi-AZ). QA: Aurora Serverless v2 (Pausa automática por inactividad).	PROD: Operador Patroni/CrunchyData garantizando HA en 3 nodos. QA: Réplica única gestionada por el mismo operador.
Almacenamiento (Archivos/PDF)	PROD/QA: Discos atados a servidores (CIR-MOD-PROY, Docker Host).	PROD / QA: Amazon S3 (Buckets separados lógicamente por entorno). Capacidad ilimitada y pago por GB consumido.	PROD / QA: Clúster MinIO On-Premise o Persistent Volume Claims (PVC) respaldados por SAN corporativa.
Utilitarios (PDF/OCR)	PROD: 1 Nodo VM Docker Host (CIR-POS-DK-CONT). QA: Compartiendo el mismo nodo PROD.	PROD / QA: Tareas delegadas a funciones <i>Serverless</i> (AWS Lambda) o Pods Fargate efímeros que solo consumen recursos al generar un PDF/OCR.	PROD / QA: Pods dedicados escalables basados en demanda transaccional (ej. KEDA scaling por longitud de cola).

B4.2 Conclusión del Dimensionamiento Estratégico

El modelo AS-IS sufre de un problema clásico de **aprovisionamiento estático**: el entorno de QA mantiene recursos encendidos 24/7 aunque solo se utilice en ventanas de prueba, mientras que Producción no puede escalar rápidamente ante picos transaccionales porque depende de la creación manual de VMs.

Valor de la Propuesta TO-BE: Al desacoplar el software del hardware mediante contenedores (Spring Boot) y orquestadores (EKS/OpenShift):

1. **En AWS:** El ambiente de **QA** puede volverse completamente elástico y efímero (escalar a 0 en las noches y fines de semana), reduciendo drásticamente el gasto en nube (*FinOps*). Producción absorberá cualquier pico de carga mediante el Auto-escalado Horizontal de Pods (HPA) en Fargate, cobrando solo por el milisegundo de cómputo consumido.
2. **En On-Premise:** El clúster OpenShift/Kubernetes consolidará toda la capacidad de cómputo en un *Pool* unificado. El planificador (Scheduler) distribuirá inteligentemente las cargas de QA y PROD (aisladas por *Namespaces*) sobre los *Worker Nodes*, maximizando el uso del hardware físico (RAM/CPU) que actualmente se encuentra ocioso o mal distribuido.